The Pollis Guide

How to use Pollis, one topic at a time

How this guide works

Each topic takes two facing pages: what the feature is for, where to find it, the steps, its limits, and what to do when something goes wrong. Read the topics in any order.

Menu paths are written **Compose > Collaboration > Join Session...**, and buttons as they appear, such as **Save As...**. A setting is named by its id, such as `pollis.collaboration.signalling`, which you can paste into the search box of **Settings**.

This guide describes Pollis as it is on 16 October 2026. What is planned but not available yet is said to be not available.

Getting Pollis

Pollis is free and open source, for Windows, macOS and Linux: github.com/saragga/pollis/releases/latest.
Questions and problems: **Help > Report Issue**, or contact@pollis.org.

Contents

2 Getting Started
4 Inside a Panel
6 Data and Databases
8 Customisation
10 Toolboxes
12 Workflows
14 New Methods
16 Reports
18 Practice and Learning Activities
20 Collaboration
22 Orchestration

© 2026 Antonio Saragga Seabra · Licensed under CC BY-SA 4.0 · Draft, 16 October 2026

Getting Started

Welcome page · Julia REPL

Installing Pollis and Julia, the first launch, and how the Julia packages a panel needs are installed.

Pollis is a single application with no account and no licence key. It writes Julia code; Julia, which runs that code, is installed separately, from the Welcome page, in a few minutes.

Install Pollis

Download the file for your system from github.com/saragga/pollis/releases/latest.

- **macOS** (Apple silicon), `Pollis-darwin-arm64.zip`:
  1. Unzip it and move `Pollis.app` to `Applications`.
  2. In Terminal, run `xattr -cr /Applications/Pollis.app`.
  3. Open Pollis.
- **Windows** (x64), `PollisSetup-x64.exe`:
  1. Double-click it. If SmartScreen warns you, choose **More info**, then **Run anyway**.
  2. Click **Install**. No administrator rights are needed.
  3. Pollis opens when the installation finishes; later, open it from the Start menu.
- **Linux** (x64), `Pollis-linux-x64.tar.gz`:
  1. Run `tar -xzf Pollis-linux-x64.tar.gz`.
  2. Run `Pollis-linux-x64/pollis`.

> **The apps are not signed yet**
>
> That is why macOS needs the `xattr` step and Windows may warn you. Download Pollis only from the link above.

Install Julia

If Julia is not installed, the Welcome page says so under **Alerts and Announcements**: *Julia is not installed*.

1. Choose **Install Julia**, then **Download and Install**.
2. Pollis installs it with juliaup, the official Julia version manager, in a terminal. The terminal says when Juliaup and Julia have been installed.
3. The Welcome page then says *Julia … is ready*, and the Julia REPL starts.

If Julia is already installed somewhere Pollis cannot find it, choose **Set Julia Path** instead, and give the path of the Julia program in the setting `julia.executablePath`.

The first start of the Julia REPL

When Pollis opens, the Julia REPL starts in the panel at the bottom. The first time, it installs DuckDB and PollisDatasets for the built-in database, and says *This happens only once*. They go into an environment of Pollis's own, `~/.pollis/julia`, which comes after yours: your own versions of these packages are used first. At later starts, PollisDatasets is updated from GitHub when there is a newer version; offline, the installed copy is kept.

The packages a panel needs

The **Powered by:** line at the top of a panel names its Julia packages. Beside it, a tick means they are all installed; **Install** means some are missing, and installs them in the Julia REPL, in your active Julia environment. You do not have to: the code a panel sends to the Julia REPL or to a notebook starts with a line such as

```
Pollis.packages("GLM", "Plots", "PollisDatasets")
```

which installs whichever of them Julia cannot load, and does nothing when all are there. Workflows, notebooks and Code Examples begin the same way. The first run of a panel can therefore take a few minutes; the next ones do not.

Updates

Pollis asks GitHub once a day for a newer release. When there is one, a badge appears on the **Manage** gear, and **Download Update** opens the file for your system in your browser: install it as above. At the first start of a new version, Pollis offers **What's New**; **Help > What's New in Pollis** opens the same notes at any time. To stop the checks, set `update.mode` to `none`.

A faster start

If you do not want the Julia REPL at every start, clear `julia.startREPLOnStartup`. It then starts the first time you send code to it.

If something goes wrong

macOS: "Pollis is damaged and can't be opened"	Run `xattr -cr /Applications/Pollis.app` in Terminal, then open Pollis again.
Windows: "Windows protected your PC"	Choose **More info**, then **Run anyway**.
***Julia is not installed*, though it is**	Choose **Set Julia Path** and give the path of the Julia program.
The first run of a panel seems stuck	Look at the Julia REPL: packages are being installed and compiled. Wait for the prompt to come back.
Install stays after installing	The tick appears once Julia has finished; it checks again every few seconds.

Not available yet

A Mac build for Intel processors, and signed apps.

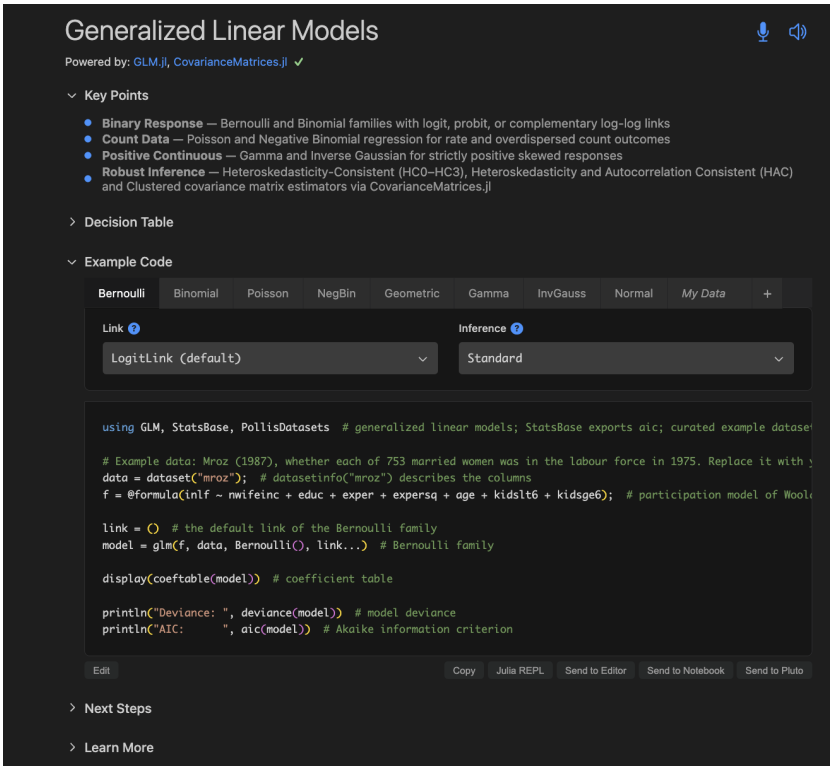
See also Inside a Panel, page 4. Data and Databases, page 6.

Inside a Panel

Explore · Model · Simulate · Optimise

What a panel is made of, and how to go from its example to code of your own.

Each item of the **Explore**, **Model**, **Simulate** and **Optimise** menus opens a *panel*: a page about one method, with an example you can run as it is, with simulated data or a dataset from PollisDatasets, and then make your own.



From top to bottom

- **The title**, and the **Powered by:** line: the Julia packages the panel uses, each a link to its documentation, with the tick or **Install** (see Getting Started, page 2).
- **Key Points**: what the method does and when to use it, in a few lines.
- **Decision Table**, in some panels: which variant to use for which data.
- **Illustration**, in some panels: a picture of the method or its variants.
- **Example Code**: the tabs of the panel's variants (above, **Bernoulli**, **Binomial**, …), its inputs, and the code. Changing an input changes the code at once.
- **Next Steps** and **Learn More**, at the bottom (next page).

Click a section's name to open or close it.

Run the example

The buttons below the code send it where you want it:

- **Julia REPL** runs it in the Julia REPL, starting it if needed.
- **Send to Notebook** opens it in a new Jupyter notebook, with the Julia kernel.
- **Send to Pluto** opens it in a new Pluto notebook, in Pollis's browser.
- **Send to Editor** opens it in a new, unsaved Julia file, and **Copy** copies it.

Code sent to the Julia REPL or to a notebook starts with the line that installs any missing package.

Make the example your own

1. Choose **Edit** below the code, and change it: your data, your variables, your options.
2. Choose **Save**, or **Cancel** to leave it as it was. A saved example is marked **Customised**, and the panel shows your version from then on.
3. To go back to the original, choose **Restore Default**, then **Confirm Restore** within four seconds. Your version goes to the Trash.

Add a tab of your own

1. Choose **+** at the end of the tabs, type a name and press **Enter**.
2. The new tab, marked **Added**, opens empty, for editing: write or paste your code, and **Save**.
3. To remove it, choose the **×** on the tab, then **Delete**; it goes to the Trash.

Your examples and tabs are files in `~/.pollis/examples`, one folder per panel. They are kept when Pollis is updated.

Next Steps

Visualise, Diagnose, Predict, Compare, Interpret: examples that carry on from the Example Code, using its results. Next Steps opens once you have run or sent the Example Code of the tab (the **?** beside it says so). Its examples have the same buttons, and can be edited and restored in the same way.

Learn More

- **Local Wikis**: a reference page on the method, in Pollis.
- **Notebook Tutorials**: notebooks that work through the method.
- **Multimedia Tutorials**: videos and talks. **Add / Edit Video** adds your own.
- **Explore References**: the papers and books behind the method, with **Open in Browser** and **Copy BibTeX to Clipboard**. **Add / Edit Reference** adds your own, from BibTeX, a link or a DOI.

> **The example data are an example**
>
> They show how the method works. To use it on your own data, change the lines that load the data, and check that the variables and options still suit your question.

If something goes wrong

Next Steps is dimmed	Run or send the Example Code of this tab first.
An error that a package is not found	Use **Julia REPL** or **Send to Notebook**, which install it, or **Install** on the **Powered by:** line.
The send buttons are greyed out	Finish editing (**Save** or **Cancel**); an added tab needs some code first.

> **Not available yet**
>
> The microphone and speaker buttons beside the title: voice comes in a later version.

See also Getting Started, page 2. Data and Databases, page 6. Customisation, page 8.

Data and Databases

Databases view · Explore > Access Files and Databases

The datasets that come with Pollis, your own databases, and the Databases view.

Pollis comes with a collection of real datasets, PollisDatasets, that its examples use, and connects to DuckDB, SQLite and PostgreSQL databases. All the work with data is done in Julia: the Databases view shows what is there and writes the code to reach it.

PollisDatasets

54 real datasets, such as `mroz`, `airpassengers`, `iris`, and `challenger`, each with its source and licence. In Julia:

- `datasets()` lists them;
- `datasetinfo("mroz")` describes one and its columns;
- `data = dataset("mroz")` loads one as a Tables.jl table.

They are installed the first time the Julia REPL starts (Getting Started, page 2). When new datasets are added, the Welcome page says so.

The built-in database

The same datasets are also a DuckDB database, *PollisDatasets*, one table per dataset, named after it, with a description of the table and of each column. It is read-only, and is rebuilt only when PollisDatasets changes. The Welcome page says when it is ready.

1. Open the **Databases** view, in the secondary side bar on the right (or choose **Open Databases View** on the Welcome page).
2. Click **PollisDatasets**, marked **built-in**. It connects in the Julia REPL as `pollis`.
3. Expand it to see its tables and their columns.

Then query it in the Julia REPL with SQL:

```
DBInterface.execute(pollis, "SELECT * FROM mroz WHERE educ > 12")
```

The Databases view

- Click a connection to connect it in the Julia REPL; **Disconnect** closes it.
- On a table, **Open in Table Viewer** shows its rows; right-click it for **Preview Rows in Julia REPL**.
- **Show Tables** and **Hide Tables**, beside a connection, expand or collapse it; **Refresh Schemas**, at the top of the view, shows tables you have just created.
- Right-click a connection for **Copy Name**, **Copy Connection Code**, **Edit Connection** and **Delete Connection**, which removes only the saved connection, never the database.

Connect to your own database

1. In the Databases view, choose **New**, then **DuckDB**, **PostgreSQL** or **SQLite**.
2. Give a **Connection Name** and the **Julia Variable** that will hold the connection, and choose whether to keep it for all your work (**Global**) or for this folder only (**Current Workspace**).
3. For DuckDB or SQLite, choose the database file (**Choose File**). A DuckDB connection with no file is an empty database in memory: it can read CSV and Parquet files in place, and is lost when you disconnect. **Read Only** protects a file from changes.
4. For PostgreSQL, give the **Host**, **Port**, **Database** and **Username**.
5. Check the **Generated Code**, then **Save Connection**, or **Connect** to save it and connect.

A connection opened by your own code can be shown in the view as well:

```
con = PollisDB.show(DBInterface.connect(DuckDB.DB, "sales.duckdb"); name = "Sales")
```

> **PostgreSQL passwords**
>
> With **Ask for Password in the REPL** (the default), you type the password in the Julia REPL when you connect: it is never written in the code, in the REPL history or in a file. Without it, the password comes from `PGPASSWORD` or `~/.pgpass`.

Panels for data

- **Explore > Access Files and Databases**: **SQL Databases** (query, import a CSV, Parquet or JSON file, save a table, export a query), **CSV Files** and **Excel Workbooks**.
- **Explore > Access Data Libraries**: Hugging Face and Kaggle datasets, Yahoo Finance, Alpha Vantage, US SEC EDGAR, FRED and the European Central Bank. A provider that needs an API key shows **Set API key** on the **Powered by:** line; the key is kept in your system's secure storage.

If something goes wrong

The view shows no tables	Choose **Refresh Schemas**. A new database stays empty until you create a table in the Julia REPL.
PollisDatasets does not connect	The Julia REPL has not started yet, or is still installing: wait for the Welcome page to say the database is ready.
Cannot write to `pollis`	The built-in database is read-only: save your tables in a database of your own.
PostgreSQL does not connect	Check the host, port and **SSL Mode**; the **Connection Timeout** is 10 seconds.

> **Not available yet**
>
> International Monetary Fund Datasets, World Bank Indicators and the Kenneth French Data Library are in **Access Data Libraries**, greyed out.

See also Getting Started, page 2. Collaboration, page 20, for sharing datasets and databases.

Customisation

Pollis Views · Settings

Showing only the menus you need, and making Pollis yours.

To be written

Pollis Views (named sets of visible menu items, Max and Min, hiding at any depth); themes and settings; extensions from Open VSX.

Toolboxes

Toolboxes

Adding a whole field to Pollis with a toolbox extension.

To be written

What a toolbox is (data-only extensions); installing and removing one; building one with the Extension Agent skill.

Workflows

Compose > Workflows

Replaying a classic paper from its own data, step by step.

To be written

What a workflow is (a full replication of a classic paper with public data); the workflows available; following one; checking results against the paper.

New Methods

Compose > New Methods

Turning a new method into a tested Julia package, with its vignette.

To be written

Create Method; method.toml, inputs and targets; Package and Publish; topic and licence; vignettes (Installed and In the Registry).

Reports

Compose > Reports

From a notebook to a LaTeX report or slides.

To be written

Report and beamer output from a notebook in the Julia REPL; per-workflow templates; optional Polish with Agent.

Practice and Learning Activities

Help > Practice · Compose > Learning Activities

Practising a topic, and writing exercises for others.

To be written

Left page, the learner: Practise a Topic (multiple choice, every answer checked). Right page, the teacher: Write an Exercise, the exercise bank.

Collaboration

Compose > Collaboration

Working with others at their own computers: a chat, and sharing code, datasets, files and databases.

A session connects up to five people, each at their own copy of Pollis: two colleagues, a student and a tutor, or a small team. Everyone keeps their own editors and notebooks; in the chat you talk, and you share what you choose, when you choose. Nobody can change anything on anyone else's computer.

What you need

- Pollis on each computer, and an Internet connection.
- A network that lets sessions through. To check, choose **Compose > Collaboration > Test Connection...** before the session, and send its results (**Copy Results**) if you need help.

Start a session

1. Choose **Compose > Collaboration > Start Side by Side Session...**
2. Type your name, as the others will see it. The chat opens beside your editor, and the invitation is copied to the clipboard.
3. Paste the invitation into an email or a message to the others. To copy it again, use **Copy Invitation** at the top of the chat, or **Compose > Collaboration > Invite People...**

You are the *host*: you admit the others, up to five in all, and when you leave the session ends for everyone.

Join a session

1. Choose **Compose > Collaboration > Join Session...**
2. Paste the invitation you received, or type its code: four groups of five letters and digits, such as `7KQ2M-....`
3. Type your name. The chat opens and says when you have joined.

The chat

Write a message and press **Enter**; **Shift+Enter** starts a new line. Paste or drag an image into the message: larger images are reduced to fit the limit of 2 MB. The **+** button in the message box adds:

- **Image**: a PNG, JPEG, GIF or WebP image, shown in the chat.
- **File**: any file up to 5 MB. The others get a card with **Save As...**; Pollis never opens or runs it. Dragging a file that is not an image into the chat shares it the same way.
- **Dataset**: a CSV file up to 5 MB (see the next page).
- **Database**: a DuckDB database up to 5 MB (see the next page).

Show Participants lists who is in the session and whether each connection is direct or through a relay.

> **The invitation is the key**
>
> Anyone with the invitation can join while the session lasts, and receives whatever is shared from then on. Send it only to the people you mean, and do not share personal or confidential data in a session whose invitation may have gone further.

Share your code

While you are in a session, a bar at the bottom of each editor and notebook shows **Share My Editor**. It sends a snapshot of what you see, unsaved changes included; a notebook is sent without its outputs. For a Pluto notebook in Pollis's browser, use the **Share My Editor** button in the browser's toolbar. Sharing again replaces the earlier snapshot. Code files and notebooks up to 2 MB can be shared.

The others see your snapshot read-only, with two buttons below it:

- **Compare with Mine**: your snapshot and their own editor side by side, line by line.
- **Copy to My Editor**: replaces the content of their own editor, or the cells of their own notebook, with yours. **Undo** brings theirs back, except in Pluto.

Share data

- **A dataset** (CSV): **+ > Dataset** in the chat, **Share Dataset** on a CSV file in the Explorer, or **Share Dataset** in the toolbar of a Julia table viewer. Pollis asks before sending. The others get a card with the first five rows and **Preview**, **Save to My Workspace** and **Load into My Julia Session**, which reads it into a Julia table named after the file.
- **A database** (DuckDB): **+ > Database**. Pollis asks before sending the whole database. The others get a card with **Save As...** and **Attach in My Julia Session**, which opens it *read-only*, as a variable named after the file, and shows its tables in the Databases view.

> **Before sharing a database**
>
> Changes made in a Julia session that still has the database open may not be in the file yet: close that connection, or run `CHECKPOINT`, before sharing. And attach only databases from people you trust: a database can hold views that read files on the computer where they are queried.

Leave or end the session

Leave Session, at the top of the chat or in **Compose > Collaboration**, leaves the session. When the host leaves, the session ends for everyone. If the host's connection drops, the others wait a minute for it to come back. After the session the chat stays open, to be read, until you close it.

How you are connected

Free public servers introduce the participants to each other; after that each pair connects directly. The servers never see the session code or anything you share. By default they are nostr relays; for networks that block those, set `pollis.collaboration.signallin` to `mqtt` before starting the session (the invitation tells the others). A relay for networks that block direct connections is not set up yet.

If something goes wrong

"Could not reach the session"	Check the code, and that the host is still in the session. Then run **Test Connection...** on both computers.
Test Connection: no introduction server reachable	The network blocks Pollis (often a firewall or proxy): try another network, such as a phone's hotspot.
Test Connection: nostr blocked, MQTT reachable	The host sets `pollis.collaboration.signalling` to `mqtt` and starts a new session.
Two people never connect	Both networks may block direct connections, which need the relay that is not set up yet: one of them tries another network.
"The session is full"	A session has at most five participants, the host included.
A file, dataset or database is refused	It is over 5 MB, or not what it says it is: send it the usual way.

> **Not available yet**
>
> **Start Same Page Session...** (editing together), **Invite an Agent...**, **Session Settings...**, **Classroom** and **Simulation Games** are in the menu but not built yet: they say so when chosen.

See also Data and Databases, page 6. Inside a Panel, page 4.

Orchestration

Compose > Orchestration · Compose > Coding Agents

Choosing and starting coding agents, and routing tasks to models.

To be written

Coding Agents (an agent-agnostic launcher); Agent Skills and Install Skills; Model Routing (tiers, and the tier for each task); Local Reasoning Models.